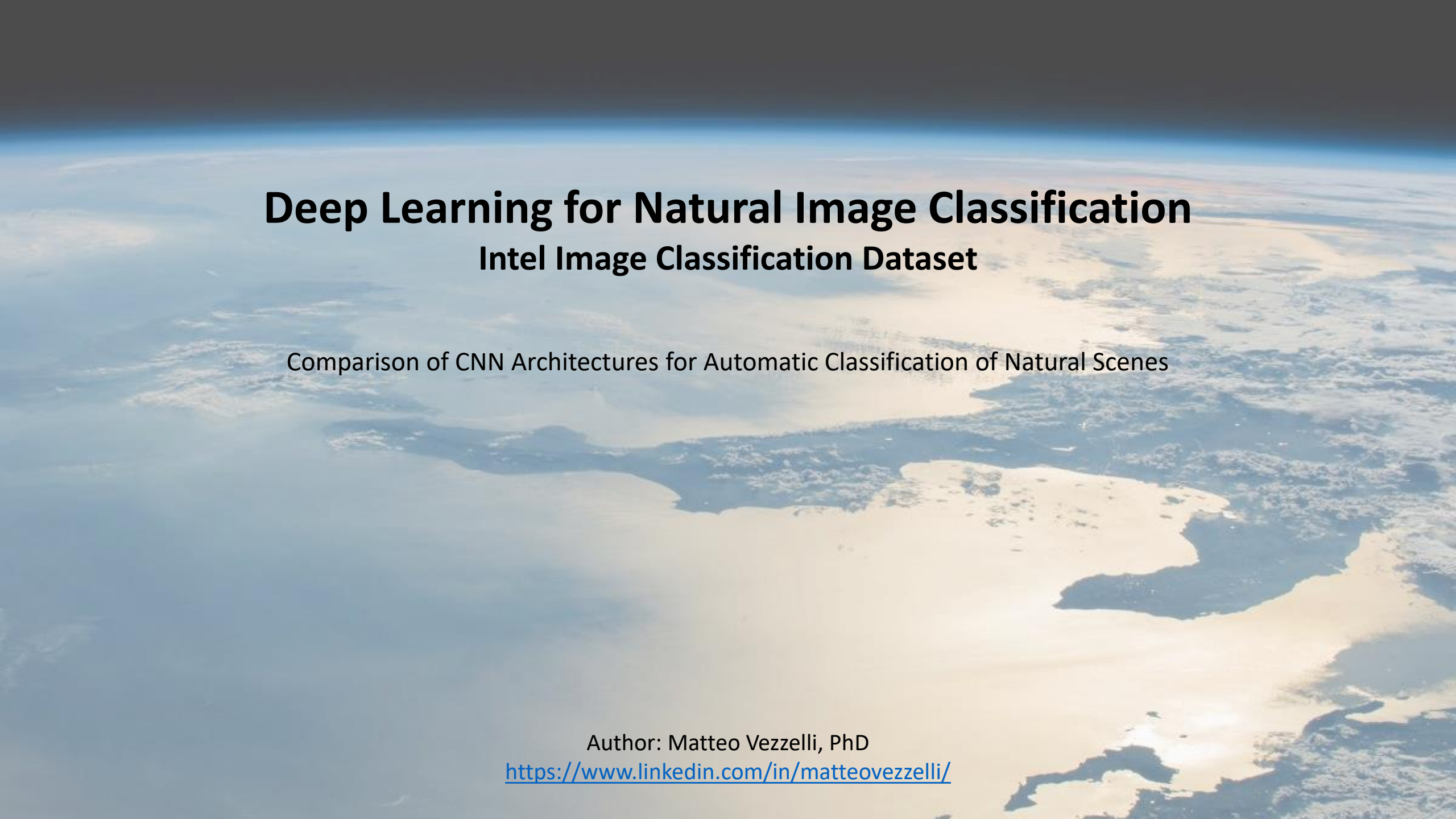


The background of the slide is a high-resolution aerial photograph of Earth from space. It shows a vast expanse of white clouds over a blue ocean, with a large landmass, likely Europe, visible in the lower right quadrant. The horizon of the Earth is visible at the top of the frame.

Deep Learning for Natural Image Classification

Intel Image Classification Dataset

Comparison of CNN Architectures for Automatic Classification of Natural Scenes

Author: Matteo Vezzelli, PhD
<https://www.linkedin.com/in/matteovezzelli/>

Project Objective

Develop and optimize **CNN models** for automatic **classification** of natural scenes

- Comparison of 3 different architectures
- Identification of the model with the best accuracy/efficiency ratio
- Practical applications: automatic geo-tagging, photographic categorization

Dataset: **Intel Image Classification** (~25,000 images, 6 classes)

<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>



Python Notebook available here: <https://github.com/mtvz42/Natural-Image-Classification/tree/main>

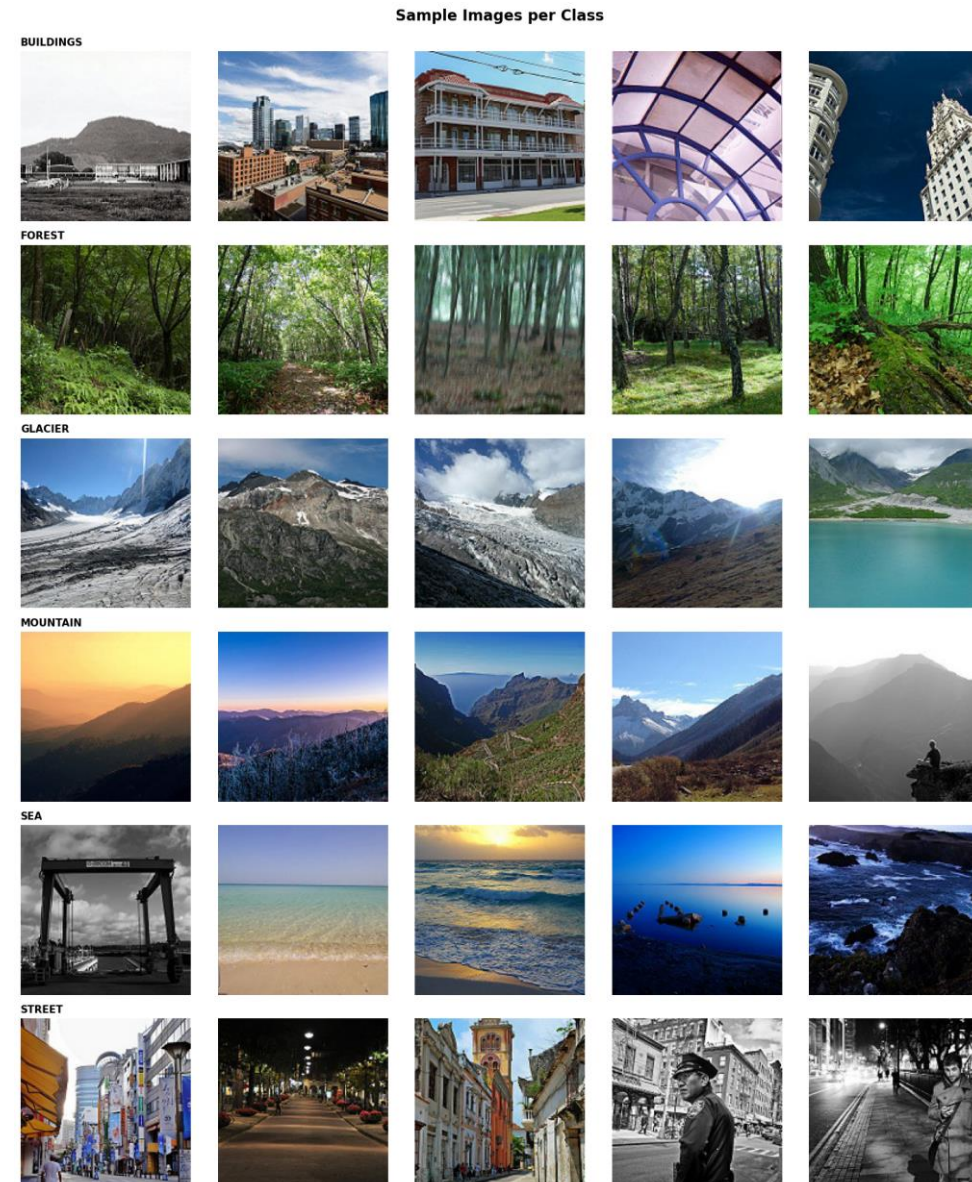
Dataset - Characteristics

Intel Image Classification Dataset

- Size: ~25,000 RGB images (150×150 pixels)
- Training set: ~14,000 images
- Test set: ~3,000 images
- Prediction set: ~7,000 unlabeled images

6 Natural Scene Classes:

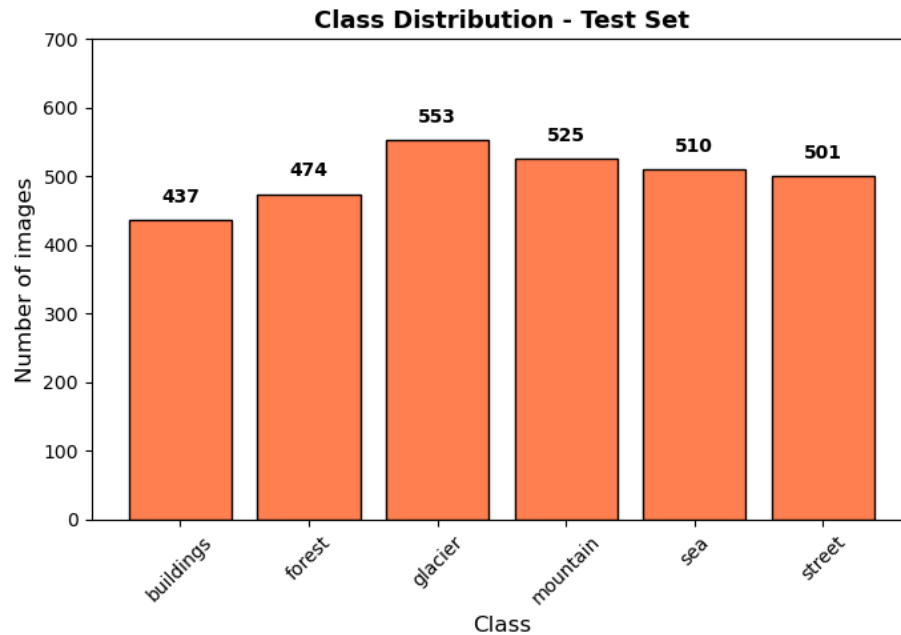
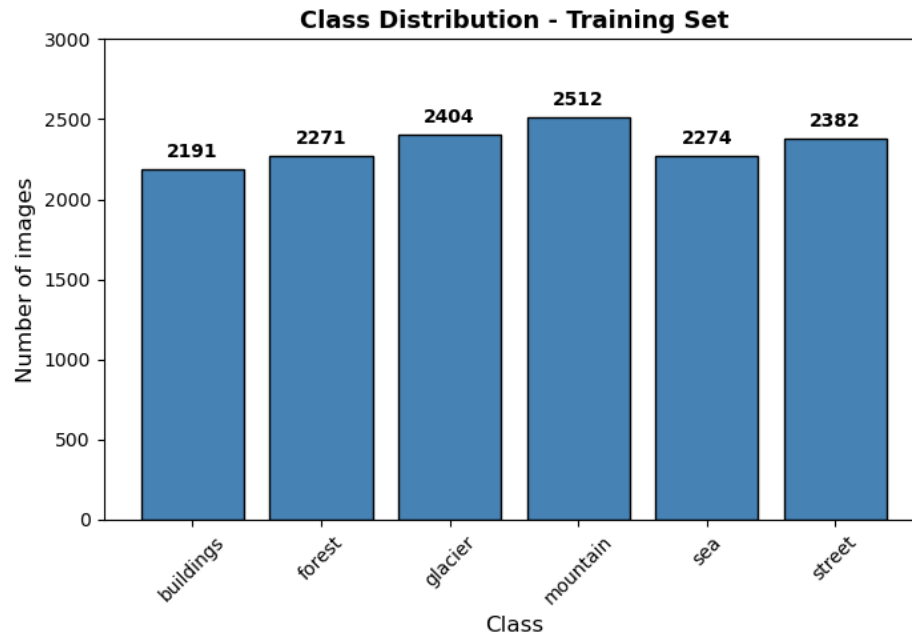
- Buildings
- Forest
- Glacier
- Mountain
- Sea
- Street



Dataset Distribution

Balanced dataset with **uniform distribution** across classes

- **Training:** ~2,300-2,500 images per class
- **Test:** ~400-550 images per class
- High intra-class variability
- Real images with natural variations



Preprocessing and Data Augmentation

Preprocessing Strategy:

- Pixel normalization: $[0, 255] \rightarrow [0, 1]$
- Resize: 96×96 pixels (computational efficiency)
- Validation split: 20% of training set

Data Augmentation (training only):

- Random rotation ($\pm 20^\circ$)
- Horizontal/vertical shift ($\pm 20\%$)
- Random zoom ($\pm 20\%$)
- Horizontal flip

Goal:

Reduce overfitting, improve generalization

Examples of Applied Data Augmentation



Architecture 1 - Base CNN

Structure:

- 3 convolutional blocks (32 → 64 → 128 filters)
- MaxPooling after each block
- Dense layer (256 units)
- Dropout (0.5)

Characteristics:

- Trainable parameters: ~3M
- Training time: ~50min (CPU), ~5min (GPU)
- Suitable for edge devices
- Simple and Fast Model

Architecture 2 - Intermediate CNN

Structure:

- 3 convolutional blocks (2 conv per block)
- Batch Normalization after each convolution
- Progressive dropout (0.25 $\rightarrow$ 0.5)
- Dense layer (512 units)

Advantages:

- More stable training
- Faster convergence
- Better gradient flow management
- Trainable parameters: $\sim 7\text{M}$
- Training time: $\sim 5\text{h}$ (CPU), $\sim 30\text{min}$ (GPU)

Architecture 3 - Transfer Learning

Structure:

- MobileNetV2 Pre-trained on ImageNet
- Custom classifier: GlobalAveragePooling → Dense(256) → Output(6)
- Transfer learning from ImageNet (1.4M images)

Advantages:

- Leverages pre-learned features
- Fewer epochs required
- Lower risk of overfitting
- Trainable parameters: ~1.3M (classifier only)
- Training time: ~4h (CPU), ~20min (GPU)

MobileNetV2: Inverted Residuals and Linear Bottlenecks

Mark Sandler Andrew Howard Menglong Zhu Andrey Zhmoginov Liang-Chieh Chen
Google Inc.
{sandler, howarda, menglong, azhmogin, lcchen}@google.com

<https://arxiv.org/abs/1801.04381>

Training Configuration

Common Parameters:

- Optimizer: Adam (learning rate = 0.001)
- Loss: Categorical Crossentropy
- Batch size: 32
- Epochs: 25 (max)
- Seed: 42 (reproducibility)

Callbacks:

- Early Stopping: patience=5, restore best weights
- ReduceLROnPlateau: factor=0.5, patience=3
- Overfitting prevention and automatic optimization

Results

Test Set Accuracy

Performance comparison on test set

Best Model: Transfer Learning

- +5.6% vs. Base CNN
- +3.7% vs. Intermediate CNN

Model Complexity

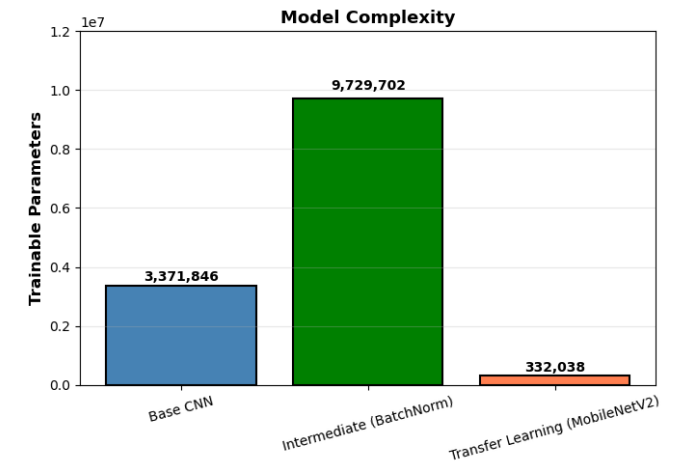
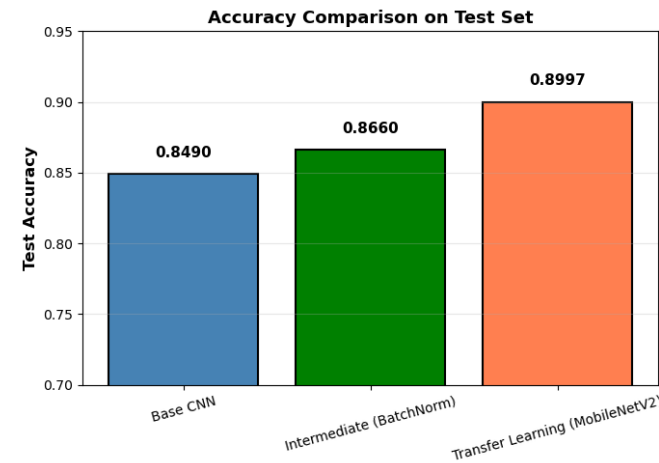
Trainable Parameters

- Base CNN: ~3.37M
- Intermediate CNN: ~9.73M
- Transfer Learning: ~0.33M

Transfer Learning: best performance/complexity ratio

- Fewer parameters to train
- Greater efficiency
- Simpler deployment

=== MODEL PERFORMANCE COMPARISON ===					
	Model	Test Accuracy	Test Loss	Total Parameters	Trainable Parameters
	Base CNN	0.849000	0.434077	3371846	3371846
	Intermediate (BatchNorm)	0.866000	0.400215	9731622	9729702
	Transfer Learning (MobileNetV2)	0.899667	0.282877	2592582	332038



Training Curves

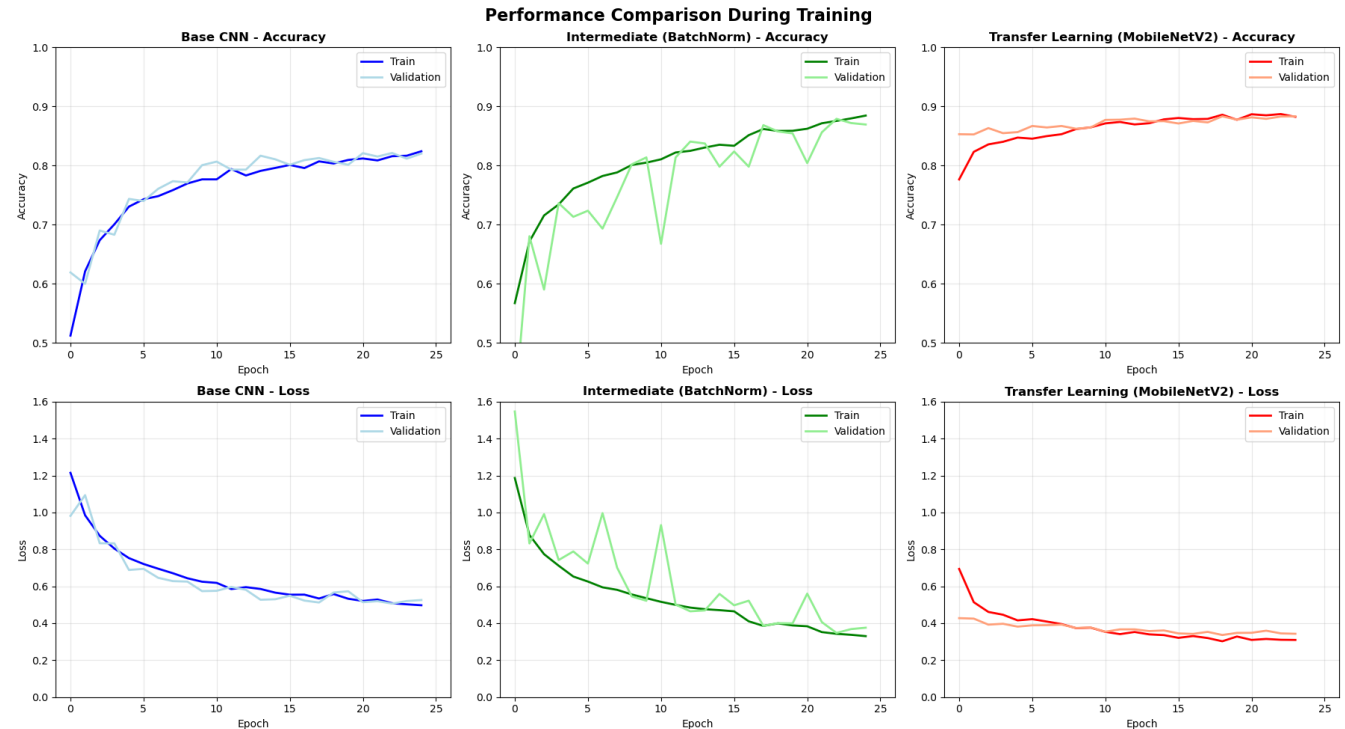
Convergence and Overfitting Analysis

Base CNN (Baseline): Exhibits the highest degree of overfitting, with a final generalization gap of approximately 8% between training and validation accuracy.

Intermediate Model (BatchNorm): Shows improved accuracy and faster loss reduction, though the validation curves are significantly more volatile compared to the other models.

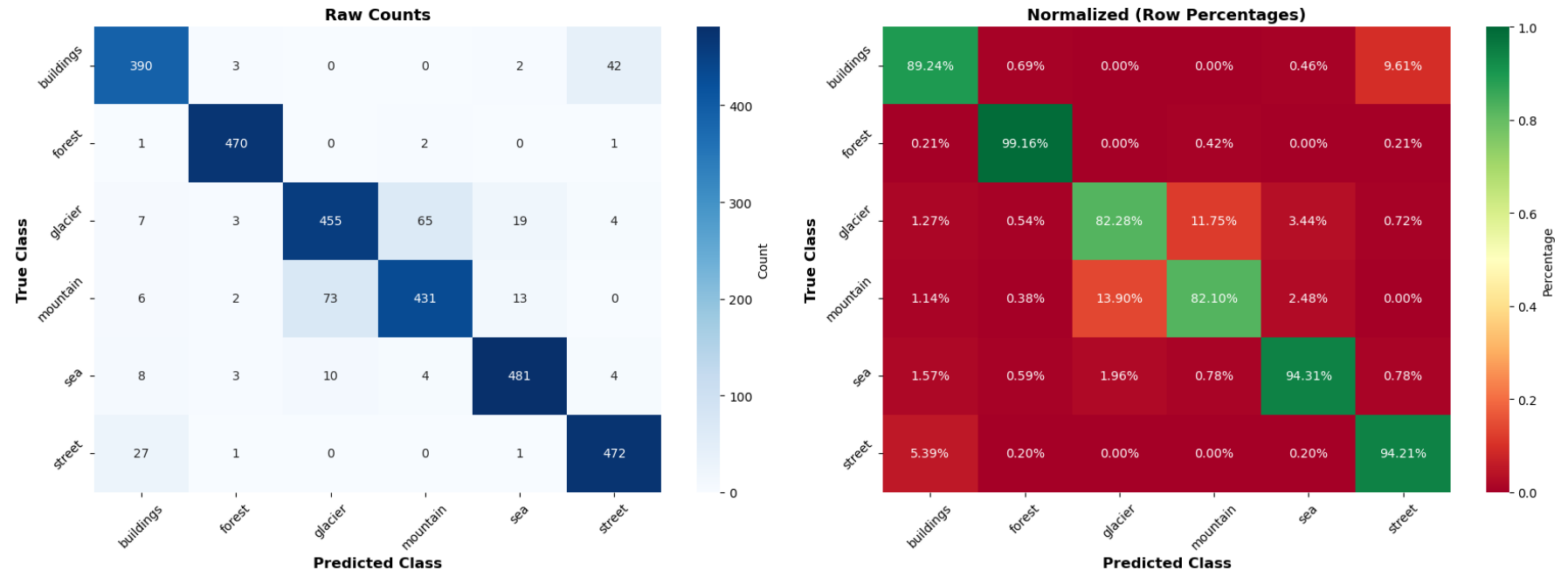
Transfer Learning (MobileNetV2): Demonstrates superior performance with the fastest convergence (stabilizing around epochs 10–12) and the best generalization, maintaining a minimal gap of ~3%.

Stability and Regularization: The inclusion of Batch Normalization and Dropout effectively stabilizes the training process and narrows the gap between training and validation metrics.



Confusion Matrix - Transfer Learning

Confusion Matrix Analysis - Best Model



High Accuracy Classes:

- Forest: 99.2%
- Sea: 94.3%
- Street: 94.2%

Main Confusions:

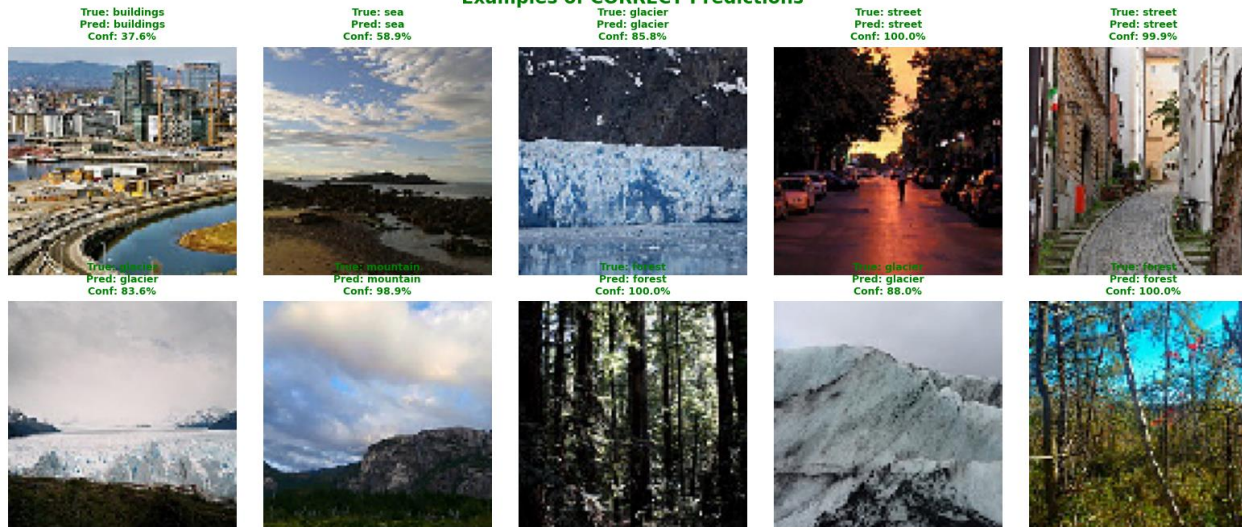
- Glacier $\leftrightarrow$ Mountain (11-14%)
- Buildings $\leftrightarrow$ Street (5-10%)
- Glacier $\leftrightarrow$ Sea (1-4%)

Causes: visual overlap, shared features, urban context

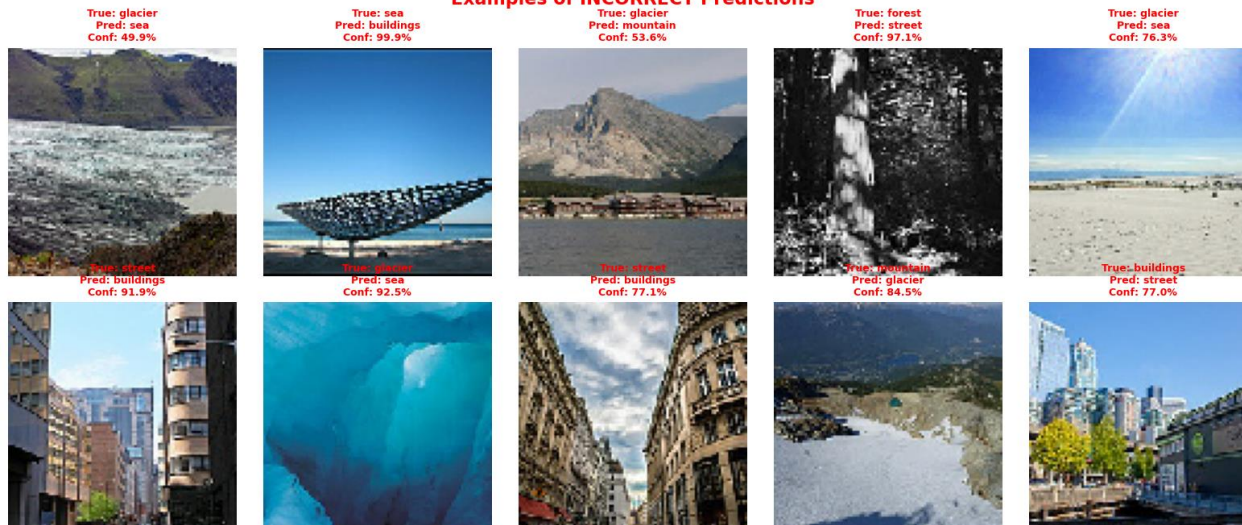
=== CLASSIFICATION REPORT ===				
	precision	recall	f1-score	support
buildings	0.8884	0.8924	0.8904	437
forest	0.9751	0.9916	0.9833	474
glacier	0.8457	0.8228	0.8341	553
mountain	0.8586	0.8210	0.8393	525
sea	0.9322	0.9431	0.9376	510
street	0.9025	0.9421	0.9219	501
accuracy			0.8997	3000
macro avg	0.9004	0.9022	0.9011	3000
weighted avg	0.8988	0.8997	0.8990	3000

Predictions - Transfer Learning

Examples of CORRECT Predictions



Examples of INCORRECT Predictions



=== PER-CLASS ACCURACY ===

```

buildings : 89.24% (390/437)
forest    : 99.16% (470/474)
glacier   : 82.28% (455/553)
mountain  : 82.10% (431/525)
sea       : 94.31% (481/510)
street    : 94.21% (472/501)
    
```

=== TOP 5 MISCLASSIFICATIONS ===

```

1. mountain → glacier : 73 times (13.9%)
2. glacier  → mountain : 65 times (11.8%)
3. buildings → street  : 42 times ( 9.6%)
4. street   → buildings : 27 times ( 5.4%)
5. glacier  → sea      : 19 times ( 3.4%)
    
```


Evaluation on unseen data - Transfer Learning

Prediction Set (seg_pred): ~7,000 images without labels

Production Environment Simulation:

- Average confidence score: 91.7%
- Median: 99.5%
- Range: 35.5% - 100%

Prediction Distribution:

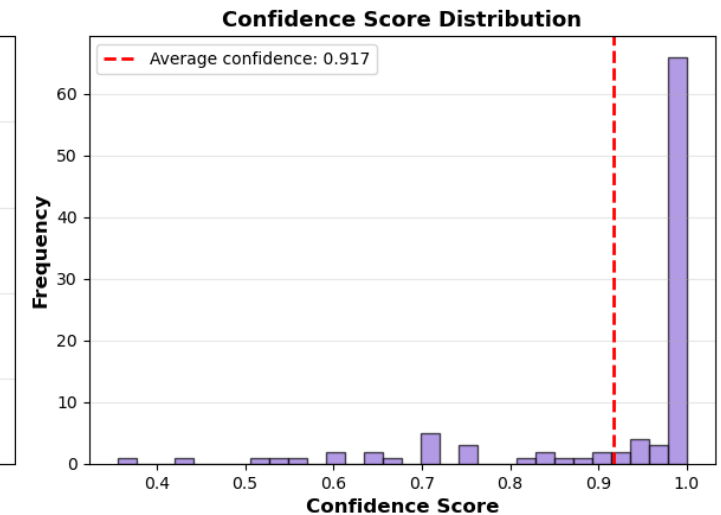
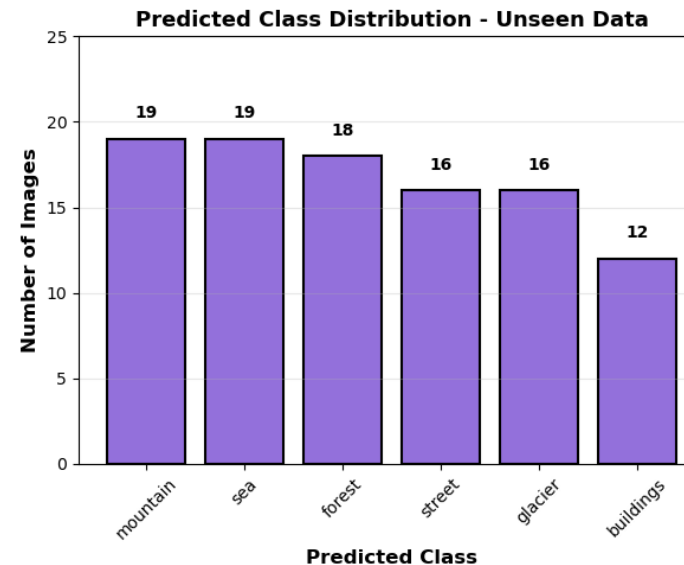
- Balanced across classes
- Consistent with training set
- No significant drift

=== PREDICTION STATISTICS ===

Average Confidence: 0.9171
Median Confidence: 0.9953
Min Confidence: 0.3553
Max Confidence: 1.0000

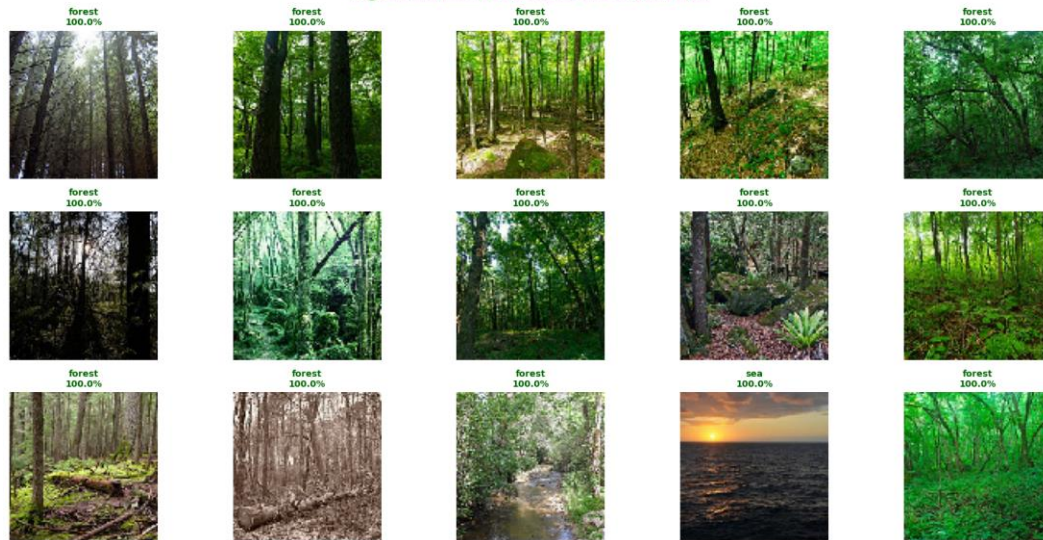
=== PREDICTED CLASS DISTRIBUTION ===

buildings	:	12 images (12.0%)
forest	:	18 images (18.0%)
glacier	:	16 images (16.0%)
mountain	:	19 images (19.0%)
sea	:	19 images (19.0%)
street	:	16 images (16.0%)



Predictions on unseen data - Transfer Learning

High Confidence Predictions on Unseen Data

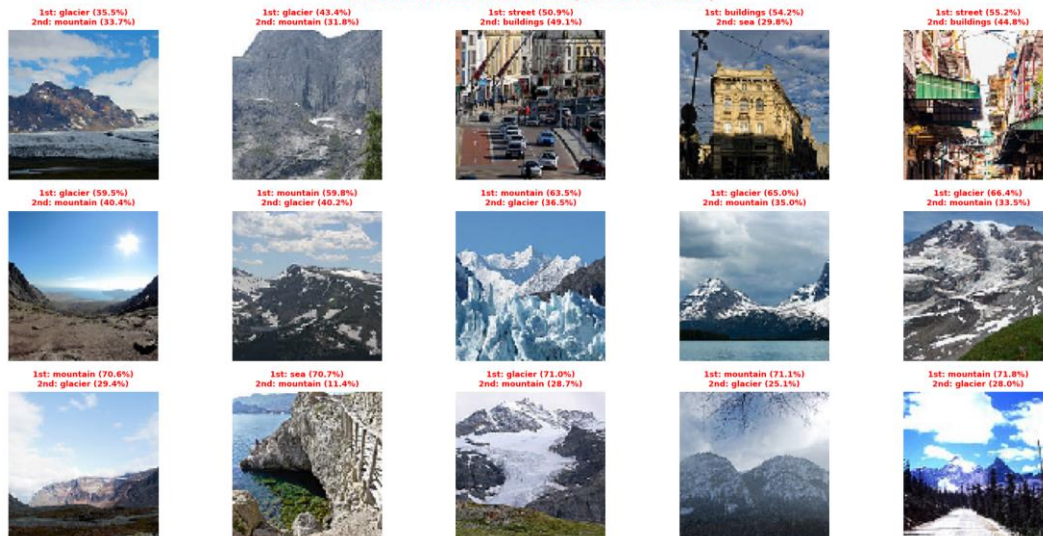


Predictions with High Confidence (>95%)

Characteristics:

- Marked distinctive features
- High-quality images
- Centered and clear subject
- Suitable for automatic tagging

Low Confidence Predictions (Uncertain Cases)



Uncertain Predictions (<70%)

Characteristics :

- Images with multiple elements
- Transitions between scenes (mountain-forest)
- Ambiguous perspectives
- Reduced image quality

Conclusions

Strategies that worked

Data Augmentation: +5-10% accuracy

- Invariance to rotation and zoom
- Overfitting reduction

Batch Normalization: +3.7% accuracy

- Stable training
- Faster convergence

Transfer Learning: +5.6% accuracy

- Leveraging ImageNet knowledge
- Less data required

Early Stopping: 5-10 epochs saved

- Auto-restore best weights
- Overtraining prevention

Limitations

Dataset and Model Constraints:

- Limited resolution (150×150 → 96×96)
- Reduced variability (mainly daytime, good weather)
- 10-15% confusion between similar classes
- Some incorrect labels in dataset
- No "unknown" class or multi-label support

Technical Gaps:

- Absence of ensemble methods
- Limited hyperparameter tuning
- Lack of interpretability (Grad-CAM)
- Validation on single dataset

Technical Improvements

Transfer Learning Fine-tuning:

- Unfreeze last 10-20 layers of MobileNetV2
- Reduced learning rate (1e-5)
- Expected gain: +2-3% accuracy

Model Ensemble:

- Combine predictions from 3 models
- Weighted voting based on validation accuracy
- Expected gain: +3-5% accuracy, greater robustness

Dataset Expansion:

- Different conditions (nighttime, rain, aerial)
- Data synthesis (style transfer, GANs)

Thank You!

Contact:

Matteo Vezzelli, PhD

<https://www.linkedin.com/in/matteovezzelli/>

Full Python notebook:

<https://github.com/mtvz42/Natural-Image-Classification/tree/main>